

Google Cloud | Gemini Enterprise

Hackathon Developer Guide: Building the FINTRAC AI Anomaly Detector

Target Platform: Gemini Enterprise, Google Cloud, BigQuery **Focus:** High-code Agent Development, ReAct Architecture, Model Context Protocol (MCP)

1. Hackathon Objective

Welcome to the FINTRAC Agent Hackathon. Your mission is to build an enterprise-grade AI agent capable of detecting financial crime. Traditional Anti-Money Laundering (AML) systems rely on rigid, rule-based SQL logic that generates massive false positives and misses sophisticated, evolving evasion tactics.

In this hackathon, you will leverage the Agent Development Kit (ADK) and Gemini Enterprise to build an intelligent agent that parses raw financial transactions, reasons over patterns using an LLM, autonomously queries a BigQuery database using the Model Context Protocol (MCP), and outputs highly contextual alerts for human compliance analysts.

Key Deliverable: A fully functioning, containerized AI agent that accurately flags "Structuring" (smurfing) and "Velocity" (rapid cross-border multi-hops) anomalies from a synthetic daily batch dataset.

2. Architecture Overview

You are building a secure, serverless pipeline on Google Cloud. This architecture provides the strict audit trail required for federal financial workloads.

Note on Data Residency: While production Canadian federal workloads should target the `northamerica-northeast1` (Montreal) region for sovereign data compliance, hackathon project environments may have quota restrictions. This guide defaults to `us-central1` to ensure availability of the Gemini 2.5 Flash model during the event.

[Synthetic Daily Batch Upload] → [BigQuery Transactions Table] ↓ [Cloud Run: Gemini High-Code Agent] ↓ (MCP Tool Execution) [BigQuery Anomalies Table] → [Looker Studio Analyst Dashboard]

- **BigQuery:** Serves as both the source of truth for raw transaction logs and the destination for flagged anomalies.
- **Gemini / Vertex AI:** Provides the ReAct reasoning engine.

- **Cloud Run:** Hosts the containerized high-code agent and MCP server.
- **Looker Studio:** The zero-code frontend where analysts review the agent's findings on an interactive geographical and flow map.

3. Prerequisites & Environment Setup

Before coding, ensure your team has access to a Google Cloud Project with the Vertex AI and BigQuery APIs enabled.

Step 3.1: Scaffold the Project

Initialize your workspace using the Agents CLI and install the required BigQuery dependencies.

Bash

None

```
# Install the Agents CLI
uvx google-agents-cli setup

# Scaffold a new high-code agent
agents-cli create fintrac-detector-USERNAME -y --deployment-target
agent_runtime
cd fintrac-detector

# Add BigQuery to the project dependencies (Critical for Cloud Run
deployment)
uv add google-cloud-bigquery
```

4. Data Engineering: Ingesting the Batch

You cannot test an AML agent without realistic data. Use this Python script to generate a daily batch of synthetic transactions.

Step 4.1: Generate Synthetic Data

Create `generate_batch.py` in your root folder:

Python

```
None
import csv
import uuid
from datetime import datetime, timedelta
import random

FILENAME = 'fintrac_batch.csv'
BASE_TIME = datetime.now()
LOCAL = ["K1A 0G9", "K2P 2N2", "K1S 5B6"]
OFFSHORE = ["KY1-1104", "BM HM 11", "VG1110"]

transactions = []

# Generate Normal Noise
for i in range(50):
    transactions.append({
        "tx_id": f"TXN-{uuid.uuid4().hex[:8].upper()}",
        "timestamp": (BASE_TIME -
timedelta(minutes=random.randint(1, 1400))).isoformat(),
        "sender": f"ACCT-{random.randint(10000, 99999)}",
        "receiver": f"ACCT-{random.randint(10000, 99999)}",
        "amount": round(random.uniform(50.0, 4500.0), 2),
        "sender_postal": random.choice(LOCAL),
        "receiver_postal": random.choice(LOCAL)
    })

# Inject Structuring Anomaly
```

```

smurf_time = BASE_TIME - timedelta(hours=2)
for i in range(3):
    transactions.append({
        "tx_id": f"TXN-{uuid.uuid4().hex[:8].upper()}",
        "timestamp": (smurf_time +
timedelta(minutes=i*45)).isoformat(),
        "sender": "ACCT-SMURF01",
        "receiver": "ACCT-TARGET99",
        "amount": round(random.uniform(9500.0, 9950.0), 2),
        "sender_postal": "K1S 5B6",
        "receiver_postal": "K2P 2N2"
    })

transactions.sort(key=lambda x: x["timestamp"])

with open(FILENAME, mode='w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=transactions[0].keys())
    writer.writeheader()
    writer.writerows(transactions)

```

Step 4.2: Load to BigQuery

Bash

None

#generate the synthetic data

```
python generate_batch.py
```

#create the BigQuery table for the source data

```
bq mk fintrac_prod-USERNAME
```

#Load the data into BigQuery

```
bq load --source_format=CSV --autodetect fintrac_prod.transactions
./fintrac_batch.csv
```

#Validate the data was properly loaded into BigQuery

```
bq query --use_legacy_sql=false 'SELECT tx_id, timestamp, amount, receiver_postal FROM fintrac_prod.transactions-USERNAME ORDER BY timestamp DESC LIMIT 50'
```

#Create the Anomalies table

```
bq mk --table \
    fintrac_prod.flagged_anomalies \
    tx_id:STRING,timestamp:STRING,source:STRING,target:STRING,
    amount:FLOAT,anomaly_type:STRING,score:FLOAT
```

#Enable the AI platform API on the project if it hasn't been enabled already

```
gcloud services enable aiplatform.googleapis.com
--PROJECT-NAME
```

5. Core Agent Development

Important: To prevent `ModuleNotFoundError` during container build, ensure your tool files and instructions are saved directly inside the `app/` folder.

Step 5.1: Build the BigQuery MCP Server (`app/mcp_bigquery.py`)

Python

```
None
from google.cloud import bigquery

client = bigquery.Client()

def query_transactions(sql_query: str):
    """Executes a read-only SQL query against the BigQuery
    transactions table."""
    if any(keyword in sql_query.upper() for keyword in ["DROP",
    "DELETE", "UPDATE", "INSERT"]):
        return "Error: Only SELECT queries are allowed via this
    tool."

    results = client.query(sql_query).result()
    return [dict(row) for row in results]

def log_anomaly(tx_id: str, timestamp: str, source: str, target:
str, amount: float, anomaly_type: str, score: float):
    """Logs a detected suspicious transaction into the
    flagged_anomalies table."""
    query = f"""
        INSERT INTO `fintrac-prod-USERNAME.flagged_anomalies`
        (tx_id, timestamp, source, target, amount, anomaly_type,
    score)
        VALUES ('{tx_id}', '{timestamp}', '{source}', '{target}',
    {amount}, '{anomaly_type}', {score})
    """
    client.query(query).result()
    return f"Successfully logged anomaly for transaction {tx_id}"
```

Step 5.2: Define the Agent Logic (`app/agent.py`)

Python

```
None
import os
import google.auth
from google.adk.agents import Agent
from google.adk.apps import App
from google.adk.models import Gemini
from google.genai import types

from .mcp_bigquery import query_transactions, log_anomaly

_, project_id = google.auth.default()
os.environ["GOOGLE_CLOUD_PROJECT"] = project_id
os.environ["GOOGLE_CLOUD_LOCATION"] = "us-central1"
os.environ["GOOGLE_GENAI_USE_VERTEXAI"] = "True"

current_dir = os.path.dirname(__file__)
instructions_path = os.path.join(current_dir, "GEMINI.md")
with open(instructions_path, "r") as f:
    system_instructions = f.read()

root_agent = Agent(
    name="fintrac_agent",
    model=Gemini(
        model="gemini-2.5-flash",
        retry_options=types.HttpRetryOptions(attempts=3),
    ),
    instruction=system_instructions,
    tools=[query_transactions, log_anomaly],
)

app = App(root_agent=root_agent, name="app")
```

Step 5.3: Define System Instructions (**app/GEMINI.md**)

Plaintext

None

You are the FINTRAC Compliance AI Agent. Analyze daily batches and flag AML patterns.

Rules:

1. Schema Discovery: Always run ``SELECT column_name FROM fintrac_prod.INFORMATION_SCHEMA.COLUMNS WHERE table_name = 'transactions'`` before your audit.
2. Structuring: Look for deposits just below \$10,000.
3. Velocity: Flag rapid hops to offshore postal codes (e.g., KY, BM, VG).
4. Use ``query_transactions`` to audit and ``log_anomaly`` to report findings.

6. Local Testing & Production Deployment

Test your agent locally using the playground. If you encounter an "address already in use" error, clear the port first.

Bash

None

```
# Run the local testing server
agents-cli playground
```

```
# Use this command to clear frozen server ports
fuser -k 8080/tcp
```

Click on Web Preview on the **top right corner** and select **app** from the drop down

Prompt for the agent:

"You are the FINTRAC Compliance AI Agent. First, use your tool to run `SELECT column_name FROM fintrac_prod.INFORMATION_SCHEMA.COLUMNS WHERE table_name = 'transactions'` to learn the exact schema of the table. Once you know the real column names, run an AML audit on the `fintrac_prod.transactions` table. Look for structuring patterns (deposits <\$10k) and rapid offshore movement. Once you find the suspicious accounts, log them to the `fintrac_prod.flagged_anomalies` table using your logging tool, making sure to map the actual table columns to the required tool parameters."

Bash

None

```
bq query --use_legacy_sql=false 'SELECT * FROM
fintrac_prod-USERNAME.flagged_anomalies ORDER BY timestamp DESC
LIMIT 10'
```

Once the agent successfully logs anomalies to BigQuery, deploy it to Cloud Run:

Bash

None

```
agents-cli deploy --project hso-hackathon
```

7. Use a Custom BigQuery Query in Looker

Instead of trying to join tables inside Looker Studio (which can be slow), we will create a flat "View" using a Custom Query when you connect BigQuery.

- Go to <https://datastudio.google.com> -> Login -> Create Report.
- Select BigQuery as your data connector.
- Choose Custom Query -> select your **USER-PROJECT** project.
- Paste a query like this to pull everything together (joining your anomalies with the original transactions to get the postal codes):

SQL Code

```
SELECT
  a.timestamp,
  a.tx_id,
  a.source,
  a.target,
  a.amount,
  a.anomaly_type,
  a.score,
  t.sender_postal,
  t.receiver_postal,
  -- Hackathon trick: Derive a 'Country' for the map based on the postal code
  CASE
    WHEN t.receiver_postal LIKE 'KY%' THEN 'Cayman Islands'
    WHEN t.receiver_postal LIKE 'BM%' THEN 'Bermuda'
    WHEN t.receiver_postal LIKE 'VG%' THEN 'British Virgin Islands'
    ELSE 'Canada'
  END as destination_country
FROM `fintrac_prod-USERNAME.flagged_anomalies` a
LEFT JOIN `fintrac_prod-USERNAME.transactions` t ON a.tx_id = t.tx_id
```

2. Building the Components 1. Geographic Risk Map (Google Maps)

Looker Studio has native Google Maps integration, but it needs recognized text (like "Canada" or "Cayman Islands") to drop the pins.

- Add Chart: Click Add a chart -> choose the Google Maps Bubble Map.
- Location Dimension: Use the new `destination_country` field we created in the custom query. (Looker might ask you to change the data type of this field to "Country" in the source settings).
- Size/Color Metric: Set the metric to Record Count or Amount (so Canada Islands gets a massive red bubble).

2. The Flow Map (Sankey Diagram)

Native Looker Studio doesn't have a Sankey diagram out-of-the-box, but you can add it easily via Community Visualizations.

- Add Chart: Click the small icon next to "Add a chart" that looks like four squares (Community Visualizations) -> click Explore more -> search for Sankey.
- Dimension 1 (Source): `source` (The sender account).
- Dimension 2 (Target): `tx_id` (Transaction_ID).
- Weight/Metric: `amount`.
- *Insight*: This will automatically draw thick flowing lines for large CAD transfers moving from account to account.

3. Analyst Audit Table

This is your standard STR (Suspicious Transaction Report) export view.

- Add Chart: Select a standard Table.
- Dimensions: `timestamp`, `tx_id`, `source`, `target`, `anomaly_type`, `destination_country`.
- Metrics: `amount`, `score`.
- Sort: Sort by `score` DESCENDING so the AI's highest confidence anomalies (0.98+) sit right at the top for the analysts.